

PHARMARIDGE DEPLOYMENT RUNBOOK

One Windows laptop. Many client accounts. No crossed wires.

Terminal-first Git SSH, Cloudflare account isolation, new D1 databases, and Admin-first client onboarding.

Use the Markdown file for copy/paste commands. Use this PDF to review the operating sequence with a deployment team.

Purpose

This runbook is for a developer or deployment operator using **one Windows laptop, VS Code, multiple GitHub accounts/SSH keys**, and **different Cloudflare accounts** for different pharmacy clients.

It assumes each client receives a separate repository, Worker, D1 database, custom domain and credentials. Do not use one client's database, Worker name, Git remote, Cloudflare token, or backup for another client.

Core rule: one client folder → one Git remote/SSH alias → one Cloudflare account context → one Worker → one D1 database.

1. Use a separate folder for every client

Create a stable root folder once:

```
New-Item -ItemType Directory -Force "C:\PharmaRidgeClients"
Set-Location "C:\PharmaRidgeClients"
```

Recommended layout:

```
C:\PharmaRidgeClients\
├── AcmePharmacy\
│   └── pharमारidge\
├── BrightCare\
│   └── pharमारidge\
└── NorthsideMedicals\
    └── pharमारidge\
```

Do not keep several clients in one mutable folder and repeatedly change `worker\wrangler.jsonc`. That is how a deploy lands in the wrong D1 database.

For a new client folder:

```
$Client = "AcmePharmacy"
$ClientRoot = "C:\PharmaRidgeClients\$Client\pharमारidge"
New-Item -ItemType Directory -Force $ClientRoot
Set-Location $ClientRoot
```

Copy/extract the downloaded PharmaRidge folder into `$ClientRoot`, then open that folder in VS Code:

```
code .
```

2. Keep a client deployment register

Create a small local record **outside Git** (for example, a password manager secure note or encrypted company record). Record only identifiers, not secrets in ordinary notes.

Client	Local folder	GitHub SSH alias	GitHub repo	Cloudflare account ID	Worker	D1 database	Domain
Acme Pharmacy	C:\PharmaRidge Clients\AcmePharmacy\pharmaridge	github-acme	acme-org/pharmaridge	stored securely	acme-pharmacy-app	acme-pharmacy-db	app.acme.example

Before every deploy, read the current row and confirm the folder, remote, Worker and D1 database agree.

3. Configure one SSH key per GitHub account

3.1 Check existing SSH keys

In Windows Terminal PowerShell:

```
Get-ChildItem "$HOME\.ssh"
```

If a key already belongs to another GitHub account, do not overwrite it.

3.2 Create a new client GitHub key, if needed

Use a key name that identifies the GitHub account, not the pharmacy database:

```
$GitHubKey = "$HOME\.ssh\id_ed25519_github_acme"
ssh-keygen -t ed25519 -a 64 -C "github-acme-deployment" -f $GitHubKey
```

Choose a passphrase. Do not use the client Owner PIN as an SSH passphrase.

Copy the public key and add it to the intended GitHub account under **Settings → SSH and GPG keys**:

```
Get-Content "$GitHubKey.pub" | Set-Clipboard
Start-Process "https://github.com/settings/keys"
```

If the repository belongs to an organisation using SSO, authorise the new key for that organisation in GitHub.

3.3 Start the Windows SSH agent and load the key

Run this once from an Administrator PowerShell if the service is disabled:

```
Set-Service -Name ssh-agent -StartupType Automatic
Start-Service ssh-agent
```

Then, from your normal VS Code terminal:

```
ssh-add $GitHubKey
ssh-add -l
```

4. Create SSH aliases in `~\.ssh\config`

One alias prevents Git from guessing which key to use for `github.com`.

```
$SshConfig = "$HOME\.ssh\config"
$KeyForConfig = ($GitHubKey -replace '\\', '/')

@"

Host github-acme
  HostName github.com
  User git
  IdentityFile $KeyForConfig
  IdentitiesOnly yes
"@ | Add-Content $SshConfig
```

Example with two GitHub accounts:

```
Host github-personal
  HostName github.com
  User git
  IdentityFile C:/Users/YourName/.ssh/id_ed25519_personal
  IdentitiesOnly yes

Host github-acme
  HostName github.com
  User git
  IdentityFile C:/Users/YourName/.ssh/id_ed25519_github_acme
  IdentitiesOnly yes
```

Test the intended identity:

```
ssh -T git@github-acme
```

GitHub normally replies with a successful authentication message followed by “does not provide shell access.” That is expected.

Do not use `git@github.com:...` for a multi-account client repository. Use the alias, for example `git@github-acme:acme-org/pharmaridge.git`.

5. Create the new client repository with the correct SSH identity

From the new client folder:

```
Set-Location "C:\PharmaRidgeClients\AcmePharmacy\pharmaridge"
```

If the downloaded folder contains the source project's old Git history and you want a new clean repository:

```
if (Test-Path .git) {  
    Remove-Item -Recurse -Force .git  
}  
  
git init  
git branch -M main  
git config user.name "Your Acme GitHub identity"  
git config user.email "your-acme-github-email@example.com"  
git add .  
git commit -m "Initial Acme Pharmacy PharmaRidge deployment"
```

Create an empty **private** repository in the intended GitHub account or organisation. Then connect it with the SSH alias:

```
git remote add origin "git@github-acme:acme-org/pharmaridge.git"  
git push -u origin main
```

Verify all three client boundaries before continuing:

```
git remote -v  
git config --local user.name  
git config --local user.email
```

Expected remote shape:

```
git@github-acme:acme-org/pharmaridge.git
```

If it says `github.com` without `github-acme`, correct it before pushing:

```
git remote set-url origin "git@github-acme:acme-org/pharmaridge.git"
```

6. GitHub CLI is separate from Git SSH

Git uses your SSH alias. GitHub CLI (`gh`) uses its own active browser/token session. You may create repositories in the browser and avoid switching CLI accounts entirely.

If you do use `gh` , check the active account before creating a repository:

```
gh auth status
```

Sign in to the intended GitHub account when required:

```
gh auth login --hostname github.com --git-protocol ssh --web
gh auth status
```

Use `gh auth switch` only after checking the account it selects:

```
gh auth switch --hostname github.com
```

The safe deployment identity is still the repository remote URL with its SSH alias. A GitHub CLI account switch does not change an existing repository remote automatically.

7. Use one temporary Cloudflare account context per terminal session

For multiple Cloudflare clients, do not rely on one permanently logged-in global Wrangler session. Use a least-privilege token created in the intended client Cloudflare account, then expose it only in the current PowerShell session.

Create a token in the client Cloudflare account with only the permissions the deployment needs:

```
Account:
- Workers Scripts: Edit
- D1: Edit
- Account Settings: Read

If deploying a custom domain/route:
- Zone: Read
- Workers Routes: Edit
- DNS: Edit only if your workflow needs Wrangler to create/change DNS
```

Record the **Cloudflare Account ID** in the secure client deployment register. Never commit the API token.

At the start of a client deployment terminal session:

```
$env:CLOUDFLARE_ACCOUNT_ID = Read-Host "Client Cloudflare account ID"

$CloudflareTokenSecure = Read-Host "Client Cloudflare API token" -AsSecureString
$CloudflareTokenBstr = [Runtime.InteropServices.Marshal]::SecureStringToBSTR($CloudflareTokenSecure)
try {
    $env:CLOUDFLARE_API_TOKEN =
    [Runtime.InteropServices.Marshal]::PtrToStringBSTR($CloudflareTokenBstr)
    npx wrangler whoami
}
finally {
    [Runtime.InteropServices.Marshal]::ZeroFreeBSTR($CloudflareTokenBstr)
    Remove-Variable CloudflareTokenSecure -ErrorAction SilentlyContinue
}
```

Keep `CLOUDFLARE_API_TOKEN` in the current terminal only while deploying. At the end of the session:

```
Remove-Item Env:\CLOUDFLARE_ACCOUNT_ID, Env:\CLOUDFLARE_API_TOKEN -ErrorAction SilentlyContinue
```

Do not place client Cloudflare tokens in `.dev.vars`, `.env`, GitHub Actions secrets without an approved CI design, a committed PowerShell file, or VS Code settings.

8. Install dependencies and configure this client's Worker/D1 binding

From the client folder:

```
Set-Location "C:\PharmaRidgeClients\AcmePharmacy\pharmaridge\worker"
npm ci
```

Choose unique names:

```
$WorkerName = "acme-pharmacy-app"
$DatabaseName = "acme-pharmacy-db"
```

Create the D1 database in the active client Cloudflare account:

```
npx wrangler d1 create $DatabaseName
$DatabaseId = Read-Host "Paste the new D1 database_id"
```

Update only the client folder's `worker\wrangler.jsonc` :

```
$ConfigFile = Join-Path $PWD "wrangler.jsonc"
$Config = Get-Content -Raw $ConfigFile

$Config = [regex]::Replace($Config, '("name"\s*:\s*)[^\s]+(\s)', ('$1' + $WorkerName + '$2'), 1)
$Config = [regex]::Replace($Config, '("database_name"\s*:\s*)[^\s]+(\s)', ('$1' + $DatabaseName + '$2'), 1)
$Config = [regex]::Replace($Config, '("database_id"\s*:\s*)[^\s]+(\s)', ('$1' + $DatabaseId + '$2'), 1)

[System.IO.File]::WriteAllText($ConfigFile, $Config, (New-Object System.Text.UTF8Encoding($false)))
Select-String -Path .\wrangler.jsonc -Pattern '"name"|"database_name"|"database_id"'
```

Commit the client configuration only to that client's private repository:

```
Set-Location ..
git add .\worker\wrangler.jsonc
git commit -m "Configure Acme Cloudflare Worker and D1"
git push
Set-Location .\worker
```

9. Deploy a new client safely

Run these commands only after checking the active folder, SSH remote, Cloudflare account, Worker name and database name:

```
Get-Location
git -C .. remote -v
npx wrangler whoami
npx wrangler d1 migrations apply $DatabaseName --remote
npx wrangler d1 migrations list $DatabaseName --remote
```

Bootstrap the secure deployment Admin, not the client Owner:

```
$env:PHARMARIDGE_ADMIN_NAME = Read-Host "PharmaRidge Admin full name"
$env:PHARMARIDGE_ADMIN_USERNAME = Read-Host "PharmaRidge Admin username"
$env:PHARMARIDGE_ADMIN_PHONE = Read-Host "PharmaRidge Admin phone (optional)"

$AdminPin = Read-Host "Choose a 12+ character Admin PIN/password" -AsSecureString
$AdminPinBstr = [Runtime.InteropServices.Marshal]::SecureStringToBSTR($AdminPin)
try {
    $env:PHARMARIDGE_ADMIN_PIN = [Runtime.InteropServices.Marshal]::PtrToStringBSTR($AdminPinBstr)
    npm run db:bootstrap:admin
    npx wrangler d1 execute $DatabaseName --remote --file .\first-admin.sql
}
finally {
    [Runtime.InteropServices.Marshal]::ZeroFreeBSTR($AdminPinBstr)
    Remove-Variable AdminPin -ErrorAction SilentlyContinue
    Remove-Item Env:\PHARMARIDGE_ADMIN_NAME, Env:\PHARMARIDGE_ADMIN_USERNAME,
    Env:\PHARMARIDGE_ADMIN_PHONE, Env:\PHARMARIDGE_ADMIN_PIN -ErrorAction SilentlyContinue
    Remove-Item .\first-admin.sql -ErrorAction SilentlyContinue
}
```


Set the client Worker's JWT secret:

```
$JwtSecret = node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
$JwtSecret | npx wrangler secret put JWT_SECRET
Remove-Variable JwtSecret
```

Deploy PWA assets and API together:

```
npm run deploy
```

Verify the exact Worker you just deployed:

```
$WorkerUrl = Read-Host "Paste the deployed workers.dev URL"
Invoke-RestMethod "$WorkerUrl/api/health"
Start-Process $WorkerUrl
```

Sign in as the deployment Admin, create the named client Owner in **Users & Branches**, then give the Owner their unique private credentials. The Owner creates all client people and operating records from there.

10. Add a client custom domain without crossing clients

For a dedicated Worker Assets app hostname, edit only this client's `worker\wrangler.jsonc` :

```
code .\wrangler.jsonc
```

Use a complete Worker custom domain, not a second app deployment:

```
"assets": {
  "directory": "../public",
  "run_worker_first": ["/api/*"]
},
"routes": [
  { "pattern": "app.acme.example", "custom_domain": true }
]
```

Deploy again:

```
npm run deploy
```

Verify:

```
$AppUrl = "https://app.acme.example"
Invoke-RestMethod "$AppUrl/api/health"
Start-Process $AppUrl
```

11. Switch between clients safely

Before every client switch:

```
# 1. Finish the current client session
Remove-Item Env:\CLOUDFLARE_ACCOUNT_ID, Env:\CLOUDFLARE_API_TOKEN -ErrorAction SilentlyContinue

# 2. Move to the next client repository – never overwrite the first one
Set-Location "C:\PharmaRidgeClients\BrightCare\pharmaridge"

# 3. Check Git identity and remote
Get-Location
git remote -v
git config --local user.name
git config --local user.email

# 4. Load only the next client's Cloudflare account/token for this terminal
$env:CLOUDFLARE_ACCOUNT_ID = Read-Host "BrightCare Cloudflare account ID"
# Read the next token securely as shown in section 7, then run:
npx wrangler whoami
```

A safe pre-deploy checklist:

```
Write-Host "Folder: $(Get-Location)"
git remote get-url origin
Select-String -Path .\worker\wrangler.jsonc -Pattern '"name"|"database_name"|"database_id"'
npx wrangler whoami
```

If any line belongs to another client, stop. Do not deploy.

12. Never share these between clients

```
GitHub SSH aliases/keys: separate by GitHub account
Git remote URL: one client repository only
Cloudflare API token: scoped to one client account only
Cloudflare account ID: one client account only
Worker name: unique per client
D1 database: unique per client
Custom domain: unique per client
JWT_SECRET: unique per client
Owner/Admin credentials: unique per person and client
Backups/exports: stay outside Git and never cross clients
```

13. Recovery checks

```
# Git remote for this folder
git remote -v

# Loaded SSH keys
ssh-add -l

# Test intended GitHub alias
ssh -T git@github-acme

# Active Cloudflare account context
npx wrangler whoami

# Current Worker/D1 binding in this client folder
Select-String -Path .\worker\wrangler.jsonc -Pattern '"name"|"database_name"|"database_id"'

# Remote D1 migration state
Set-Location .\worker
npx wrangler d1 migrations list $DatabaseName --remote
```

If you accidentally deploy to the wrong account, stop further changes, preserve logs, identify the Worker/D1/database names from Wrangler output, and correct the account/token/folder context before doing any remediation. Do not run a reset command as a first reaction.